

Structure

I will start by discussing the points mentioned in the Emails (see Section 1), then I will start by describing the overall approach I discussed in Thursday's meeting. And while doing so, relating it to the points raised by Prof. Monni and Ahmad (see Section 2). After that I will propose the tasks division (see Section 3).

1 Discussing Email Points

- Yes we are on the same page that each $K_{(i,j)}^{(k)} = K^{(k)}(f^{(k)}(x_i^{(k)}), f^{(k)}(x_j^{(k)}))$ where $f^{(k)}$ is non linear mapping of the k 'th feature space. And then to get the entry $K_{(i,j)} = \sum_{k=1}^p \alpha_k K_{(i,j)}^{(k)}$. This α_k is what ahmad was referring as the weight associated with each kernel, where with training some of them might go to zero, **IF WE USE appropriate methods to do so** (one example is using L1 regularization on the weights α_k , similar to how in linear regression with a lasso penalty some of the weights of the features can go to zero), and so we can start using our architecture to as a feature selection algorithm. I need to emphasize the point that at before taking any step in our Optimization procedure, we need to compute the Gram matrix K using the current values of the weights α_k and the current hyperparameters of each kernel $K^{(k)}$, this to ensure that $K_{(i,j)}$ is the same global mapping $\forall(i, j)$, notice how from this construcion we are guarrantying that also $f^{(k)}$ is the same $\forall(i, j)$ within the k 'th feature space. In our presentation we need to emphaize this point
- Notice how while constructing K we can only construct $K_{(i,j)}$ and then set $K_{(j,i)} = K_{(i,j)}$ as mayssa and prof monni pointed out.
- Now regarding the point about adding a jitter term to the diagonal of K , this to ensure numerical stability especially in pytorch where we will be inverting K . BUT I am interested about knowing this point that professor monni mentioned and I quote "Notice that adding instead a positive number $K(i,j)=K(i,j)+\epsilon$ for all i, j need not lift the eigenvalues from zero to positive." I noticed that numerically it is true, but why is true mathematically? I would like to understand this point better. I understood adding a jitter but why only on the diagonal?
- Regarding the PCA point made by ahmad, I liked the idea of it, I think this might be an innovation in the field. I think for experimentation purposes (i.e. what we would be presenting saturday, we can mention it as a future work), in my opinon focusing now on getting something working first is more important. Also I do not think there is a need to compare what we did with other methods, in my opinion focusing now on toy models and presenting these on saturday is more important.
- Another question I have Professor Monni Mentionned " $y_i = x_i + \epsilon_i$ is more difficult than havig the toy model $y_i = f(x_i) + \epsilon_i$ "
- I think the point of interpretaibility is very nice, we can even show the graphs in the presentation, but I think this is not a trivial point to make, because we need to construct appropriate toy models to illustrate this point well.

2 Overall Approach: Gaussian Processes and Bayesian Optimization

In my approach I will try to focus on two things: 2.1 training kernels in the setup of Gaussian processes, and the second one 2.2 the point that Ahmad raised (in section 3.1 of his email) “Can our kernel imitate known kernels? ”

2.1 Gaussian Processes Setup

So here I will describe the generation process (we are nature) and then the training process (we are the scientists trying to learn given these data only).

Generating the Data (Nature)

In my setup I will assume that the data is generated as follows:

$$y = f(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

I will generate the matrix X with n samples and p features. (I will use a small p like professor monni suggested, around 5 to 10) and σ_2 to be small to have a high signal to noise ratio. (side note after training is done we can draw a graph of the loss vs different values of SNR to show how the model degrades with increasing noise). The generation of X can be done using a mv or even any other distribution, because the distribution that we really care about is the one of $f(x)$ which will be in my toy model a multivariate normal distribution with a known kernel K_{true} and mean zero. Why I chose a mv for $f(x)$ because I want the assumption of GP (in training) to be actually true. Later one we can try and violate this. **Formal Setup:**

Let $X = [x_1, x_2, \dots, x_n]^T$ be the input data matrix where each $x_i \in \mathbb{R}^p$. We assume that the function values $f = [f(x_1), f(x_2), \dots, f(x_n)]^T$ are drawn from a multivariate normal distribution:

$$f \sim \mathcal{N}(0, K_{true})$$

where K_{true} is the true kernel matrix defined by a known kernel function (RBF, linear, ...) evaluated at the input points. More specifically, the entries of K_{true} are given by:

$$K_{true}(i, j) = k_{true}(x_i, x_j)$$

for a chosen kernel function k_{true} . Again k_{true} must be the SAME kernel $\forall(i, j)$.

Now that we have the function values f , the observed outputs y are then generated by adding Gaussian noise to the function values:

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

where σ^2 is the noise variance (a small value to ensure high SNR for now).

Training the Model (Scientists)

In the training phase, we will assume a Gaussian process prior over the function $f(x)$ with a kernel K that is a weighted sum of multiple kernels as discussed earlier:

$$K(x_i, x_j) = \sum_{k=1}^p \alpha_k K^{(k)}(f^{(k)}(x_i^{(k)}), f^{(k)}(x_j^{(k)}))$$

For now I will assume always that $\alpha_k = 1$ (i.e. not trainable parameters, future experiments I will start including them as trainable (relating back to the feature selection point made earlier)). The only trainable parameters will be the hyperparameters of each kernel $K^{(k)}$ (also for now the $f^{(k)}$ will be the identity function). The training objective will be to maximize the log marginal likelihood of the observed data y given the inputs X (given by equation 4 in NKN paper):

$$\operatorname{argmax}_{\theta} -\frac{1}{2}y^T(K + \sigma^2 I)^{-1}y - \frac{1}{2}\log |K + \sigma^2 I| - \frac{n}{2}\log 2\pi$$

where θ represents the hyperparameters of the kernels and σ^2 is the noise variance. I will use ggradient descent no fancy optimizers for now to optimize this objective with respect to the hyperparameters θ .

- **Question:** In the training do I include the true value of σ^2 used in the data generation or do I treat it as a hyperparameter to be learned as well? Or not account for it so directly in the model? (i.e. just use K instead of $K + \sigma^2 I$ in the above equation)

Again here while training I will keep on observing the eigenvalues of K to ensure numerical stability, and add jitter to the diagonal if needed.

After training is done I will compare the learned kernel matrix K with the true kernel matrix K_{true} used in data generation using appropriate metrics (Frobenius norm, MSE ...etc). This will help us assess how well our model has learned the underlying kernel structure of the data.

2.2 Mitigating other Kernels

This is what ahmad raised in his email (section 3.1), and similar to the first dummy example I introduced in Thursday's meeting, in Thursday's meeting I was trying to replicate a linear kernel, I will extend this to more sophisticated kernels like RBF, periodic ...etc. Back then I was using the MSE between the true kernel matrix and the learned kernel matrix as a loss function to optimize the hyperparameters of the kernels in the weighted sum. I can now incorporate the loss that Professor Monni suggested (frobenius norm between the two kernel matrices). **Formal Setup:**

The setup here will be slightly different than before. Because now the input data will be of n^2 samples (each sample corresponds to a pair (x_i, x_j) from the original data), and the output will be the corresponding entries of the true kernel matrix $K_{true}(i, j)$. So the input data matrix will be:

$$X_{pairs} = [(x_1, x_1), (x_1, x_2), \dots, (x_n, x_n)]^T$$

with n^2 samples, and the output vector will be:

$$y_{kernel} = [K_{true}(1, 1), K_{true}(1, 2), \dots, K_{true}(n, n)]^T$$

The model will then learn to predict the kernel values using the weighted sum of kernels as before (but again here for now I will keep $\alpha_k = 1$ and only train the hyperparameters

of each kernel, also for now the $f^{(k)}$ will be the identity function). So the learned kernel matrix entries will be:

$$K_{learned}(x_i, x_j) = \sum_{k=1}^p \alpha_k K^{(k)}(f^{(k)}(x_i^{(k)}), f^{(k)}(x_j^{(k)}))$$

The training objective will be to minimize the Frobenius norm between the learned kernel matrix $K_{learned}$ and the true kernel matrix K_{true} , or the MSE between the corresponding entries:

$$\operatorname{argmin}_{\theta} \|K_{learned} - K_{true}\|_F^2$$

where θ represents the hyperparameters of the kernels. Again I will use gradient descent to optimize this objective.

3 Task Division

This is my proposed task division (waiting for your feedback), and we can meet on **Monday to discuss the progress of each of us**.

Mayssa will focus on the interpretability aspect (point 3 mentioned in Section 3 of Ahmad's email). Ahmad will continue working on the classification setup (point 2 mentioned in Section 3 of Ahmad's email). I will handle point 1 from Section 3 of Ahmad's email, incorporating it into the Gaussian process setup discussed earlier in Section 2.